Avoir un environnement Python fonctionnel pour la data science

Lino Galiana
Version du 16 octobre 2026

Ce chapitre introduit les bases de l'environnement `Python` pour la *data science* en mettant l'accent sur la modularité du langage et l'utilisation des *notebooks* `Jupyter`. Il présente les éléments essentiels pour configurer un environnement Python fonctionnel, explique les avantages des IDE comme `VSCode`, et propose une prise en main des *notebooks* interactifs. Ce chapitre aborde également la gestion des erreurs et l'importance de la formation continue en `Python`, en fournissant des ressources utiles pour rester à jour dans cet écosystème dynamique.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction **1**
2 Les ingrédients pour avoir un environnement `Python` fonctionnel **3**
2.1 L'interpréteur `Python` 3
2.2 L'environnement `Python` 4
3 Utilisation des *notebooks* dans le cadre de ce cours **7**
4 Exercice pour découvrir les fonctionnalités basiques d'un *notebook* **7**
5 Comment résoudre les erreurs ? **8**
6 Continuer à se former après ce cours **9**
6.1 Contenu sur ce site 9
6.2 La veille technique 10
Informations additionnelles 10

Compétences à l'issue de ce chapitre

- Comprendre les enjeux liés à la modularité de `Python` ;
- Comprendre l'environnement quotidien de travail des *data scientists* à savoir le *notebook* ;
- Savoir quelle démarche adopter quand on rencontre une erreur ;
- Comprendre les enjeux d'une formation continue en `Python` et découvrir quelques bonnes ressources pour cela.

1 Introduction

La richesse des langages *open-source* vient de la possibilité d'utiliser des *packages* développés par des spécialistes. `Python` est particulièrement bien doté dans le domaine. Pour caricaturer, on lit parfois que `Python` est le deuxième meilleur langage pour toutes les tâches, ce qui en fait le meilleur langage.

En effet, la malléabilité de `Python` fait qu'on peut l'aborder de manière très différente selon que l'on est plutôt *SysAdmin*, développeur web ou *data scientist*. C'est ce dernier profil qui va ici nous intéresser.

Cette richesse est néanmoins un défi pour se lancer dans l'apprentissage de `Python` ou la réutilisation de codes écrits par d'autres personnes. Les langages statistiques propriétaires sont dans un modèle *top-down*, où, une fois une licence acquise, il suffit d'installer le logiciel et suivre la documentation écrite par l'entreprise développeuse pour avancer. Dans les langages *open source*, le fonctionnement sera plutôt *bottom up* : l'écosystème est enrichi par des contributions et de la documentation issue de divers horizons. Plusieurs manières de faire la même chose coexistent et cela fait partie du travail de prendre le temps avant de choisir la meilleure approche.

Il existe principalement deux manières d'utiliser `Python` pour la *data science*[1]:

1. **Installation locale** : Installer le logiciel `Python` sur votre machine, configurer un environnement adapté (il est recommandé d'utiliser `uv`), et utiliser un logiciel de développement, tel que `Jupyter` ou `VSCode`, pour écrire et exécuter du code `Python`. Nous reviendrons dans la prochaine section sur ces trois niveaux d'abstraction.
2. **Utilisation d'un environnement en ligne** : Accéder à un environnement `Python` préconfiguré via votre navigateur, hébergé sur une machine distante. C'est cette machine qui exécutera le code que vous éditez depuis votre navigateur. Cette méthode est particulièrement recommandée pour les débutants ou pour ceux qui ne souhaitent pas se soucier de la configuration du système.

La première approche présente l'inconvénient, du point de vue pédagogique, de devoir introduire de nombreuses notions, dont certaines sont complexes, avant d'écrire la moindre ligne de code. Elle rend aussi les résultats moins reproductibles car il s'agit d'une porte ouverte à des configurations personnelles, qui peuvent venir affecter l'expérience. A l'inverse, la deuxième approche présente l'intérêt de normaliser l'expérience en proposant un cadre unique pour rentrer dans un cours.

Cette deuxième approche est liée à des services *cloud*. Dans le cadre de ce cours, comme cela sera expliqué ultérieurement, nous proposons deux solutions :

- Utiliser le `SSPCloud`[2] est obligatoire pour les élèves de l'ENSAE et de l'ENSAI, et recommandé pour tous les autres. Il s'agit d'un *cloud* développé par l'administration française et mis à disposition gratuitement aux étudiants, chercheurs et agents publics.
- Pour les malheureux ne disposant pas d'un accès au `SSPCloud`, il est possible d'utiliser `Google Colab` qui met à disposition des ressources de calcul. Cependant, l'expérience utilisateur est assez médiocre, *Google* n'ayant pas vraiment amélioré le service (sauf en ajoutant de l'IA partout...) depuis le covid. Si vous avez une installation personnelle de VSCode, il sera sans doute plus agréable d'ouvrir les notebooks par ce biais. Il est également possible d'ouvrir les *notebooks* dans `Github codespaces` mais le résultat n'est pas garanti: cette solution n'a jamais vraiment été testée !

Les deux méthodes sont décrites succinctement ci-dessous. La méthode d'installation locale est présentée pour introduire des notions utiles, mais n'est pas détaillée car elle peut être complexe et sujette à des problèmes de configuration. Si vous choisissez cette voie, préparez-vous à rencontrer des difficultés techniques et des configurations parfois obscures, nécessitant souvent des recherches fastidieuses et des solutions hasardeuses pour résoudre des problèmes que vous n'auriez probablement pas rencontrés dans un environnement en ligne.

[1] Une troisième manière d'utiliser `Python`, encore en développement, s'appuie sur une technologie appelée *WebAssembly*. Cette approche permet d'exécuter du code `Python` directement dans le navigateur, offrant une expérience d'exécution locale sans nécessiter d'installation complexe.

[2] L'utilisation de cette infrastructure est obligatoire pour les élèves de l'ENSAE et ENSAI ! Pourquoi ? D'abord car elle vous permettra de dépasser les limites de vos ordinateurs personnels et de Colab pour la suite de votre scolarité. De plus, elle vous permet d'être tous assurés d'avoir les résultats attendus lors de la rédaction du TD.

2 Les ingrédients pour avoir un environnement `Python` fonctionnel

Comme évoqué précédemment, l'utilisation de `Python` en local pour la data science repose sur trois éléments principaux :

1. L'interpréteur `Python`,
2. Un environnement virtuel
3. Un environnement de développement intégré (IDE).

Chacun de ces composants joue un rôle complémentaire dans le processus de développement.

2.1 L'interpréteur `Python`

Il s'agit du langage de programmation lui-même. Installer `Python` sur votre machine est la première étape essentielle, car cela fournit l'interpréteur nécessaire pour exécuter votre code `Python`. Cette installation définit la version de `Python` utilisée. A ce stade, ce n'est que le langage de base et un outil en ligne de commande.

> **i** Illustration
>
> Si vous avez accès à une ligne de commande dans un environnement où `Python` est disponible et bien configuré (notamment ajouté au `PATH`), vous pouvez déjà faire du `Python` par le biais de la ligne de commande[3]
>
> ```
> python --version
> python -c "print(3+3)"
> echo "print(3+3)" > example.py && python example.py && rm example.py
> ```
>
> **Ligne 3**
>
> Un exemple d'utilisation de création de fichier `example.py` depuis la ligne de commande et d'utilisation de celui-ci par la ligne de commande. Créer des fichiers depuis la ligne de commande n'étant pas un mode de travail réaliste, nous aurons l'occasion d'expliquer comment créer des fichiers par le biais d'un éditeur de texte.

Cette utilisation en ligne de commande pourrait aussi être faite de manière interactive. Si vous lancez, depuis votre terminal, la commande `python` (sans nom de fichier derrière), vous devriez voir s'afficher une console commençant par un message du type

```
$ python

Python 3.13.13 (main, Aug 17 2026, 02:38:26) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

qui vous informe que vous avez lancé le logiciel `Python` dans une certaine version (ici la `3.13.13`). Vous pouvez taper directement dans celle-ci des commandes, par exemple, `3+3` qui devrait vous donner un résultat peu suprenant.

[3] Vous n'avez pas accès directement à la ligne de commande avec `Google Colab`, seulement de manière détournée par le biais de l'interface du *notebook* en faisant précéder la commande d'un `!`. Sur les services `VSCode` du `SSPCloud`, l'interface recommandée pour ce cours, vous avez accès à une ligne de commande en cliquant sur `☰ > Terminal > New Terminal`. Sur une installation personnelle de `VSCode`, ce sera en haut dans le menu `Terminal > New Terminal`.

2.2 L'environnement `Python`

Le langage `Python` est construit, comme les autres langages *open source*, sous forme de socle de base et de *packages* supplémentaires. Ce sont ces derniers qui forment l'écosystème foisonnant et dynamique de `Python` et qui permettent à ce langage d'être si confortable.

`Python` est un langage *open source* ce qui signifie que n'importe qui peut proposer la réutilisation de son code sous la forme de *package*. Il existe donc des plateformes centralisant des packages communautaires. La principale est `PyPI` dans l'écosystème `Python`.

On parle d'environnement pour désigner l'ensemble des packages disponibles à `Python` pour effectuer des traitements spécifiques. `Python` n'est pas installé avec l'ensemble des *packages* disponibles sur `PyPI`, c'est donc à vous d'enrichir votre environnement en installant de nouveaux *packages* lorsque vous en avez besoin.

Si votre `Python` est correctement configuré, vous pourrez installer de nouveaux packages avec `pip install`[4]. Nous allons utiliser de nombreux *packages* dans ce cours, cette commande reviendra donc régulièrement.

Ensuite, une fois installé, un script doit déclarer un *package* avant de l'utiliser (sinon `Python` ne saura pas où chercher telle ou telle fonction). C'est l'objet de la commande `import`. Par exemple `import pandas as pd` permet d'utiliser le package `Pandas` (à condition qu'il soit déjà installé)

> **i** Illustration
>
> Voici une illustration de la manière dont fonctionne la gestion de *packages* en `Python`
>
> ```
> python -c "import geopandas as gpd"
> pip install geopandas
> python -c "import geopandas as gpd ; print(gpd.GeoDataFrame)"
> ```
>
> **Ligne 1**
> Cela devrait provoquer une erreur si `GeoPandas` n'est pas installé `ModuleNotFoundError: No module named 'geopandas'`
>
> **Ligne 3**
> L'erreur devrait avoir disparue

> **!** Important
>
> Il est déconseillé d'utiliser des importations globales sous la forme `from pkg import *`. Par exemple, prenons deux modules qui proposent une fonction `sqrt`:
>
> ```
> # Don't do that please !
> from numpy import *
> from math import *
> sqrt([4, 3])
> ```
>
> Ces importations globales chargent toutes les fonctions et variables des modules `numpy` et `math` dans l'espace de noms global (*namespace*), ce qui peut entraîner des conflits de nommage.
>
> D'abord, cela peut mener à des résultats imprévisibles car si les implémentations diffèrent, comment sait-on si c'est la fonction du package `numpy` ou `math` qui a été utilisée. Ici ce sera celle

[4] Plus d'éléments sur le sujet des environnements sont disponibles dans le cours de 3e année « Mise en production de projets data science », notamment les différents types d'environnements virtuels (`conda` ou `venv`) et les implications de ceux-ci sur les chaînes informatiques en `Python`.

du package `math`, importé en dernier, ce qui provoquera une erreur: le module `math` ne gère que des entiers, pas des vecteurs, *a contrario* de `numpy`.

Ensuite, cela rend le code moins lisible, car il devient difficile de savoir d'où provient chaque fonction ou variable utilisée, ce qui complique la maintenance et le débogage.

Il donc préférable d'importer uniquement les fonctions nécessaires ou d'utiliser des alias explicites, comme import numpy as np et import math pour éviter ces problèmes.

2.2.a Environnements de développement et notebooks

L'utilisation de `Python` en ligne de commande est fondamentale dans le monde applicatif. Néanmoins, ce n'est pas pratique d'écrire directement son code dans la ligne de commande au quotidien. Heureusement, il existe des éditeurs adaptés qu'on appelle des **IDE** (*Integrated Development Environment*).

Ce sont des logiciels qui offrent une interface pratique pour écrire et exécuter votre code. Ils offrent des fonctionnalités permettant de simplifier la lecture et l'écriture de code: coloration syntaxique, autocomplétion, débuggage, diagnostics de qualité formelle du code, etc.

Les *notebooks* `Jupyter`[5] offrent une interface interactive qui permet d'écrire du code `Python`, de tester celui-ci et d'avoir le résultat en dessous de l'instruction plutôt que dans une console séparée. Les *notebooks* `Jupyter` sont incontournables dans les domaines de la *data science* et de l'enseignement et de la recherche car ils simplifient grandement l'exploration et le tâtonnement.

Ils permettent, dans un même document, de combiner du texte au format `Markdown` (un format de texte au balisage plus léger que `HTML` ou $\LaTeX$), du code `Python`, et du code `HTML` pour les visualisations et animations.

[5] `Jupyter` est né du projet `IPython`, un environnement interactif pour `Python` développé par Fernando Pérez en 2001. En 2014, le projet a évolué pour supporter d'autres langages de programmation en plus de Python, ce qui a conduit à la création du projet Jupyter. Le nom « Jupyter » est un acronyme qui fait référence aux trois langages principaux qu'il supporte : `Julia`, `Python` et `R`. Les *notebooks* `Jupyter` sont incontournables dans les domaines de la *data science* et de l'enseignement et de la recherche car ils simplifient grandement l'exploration et le tâtonnement.

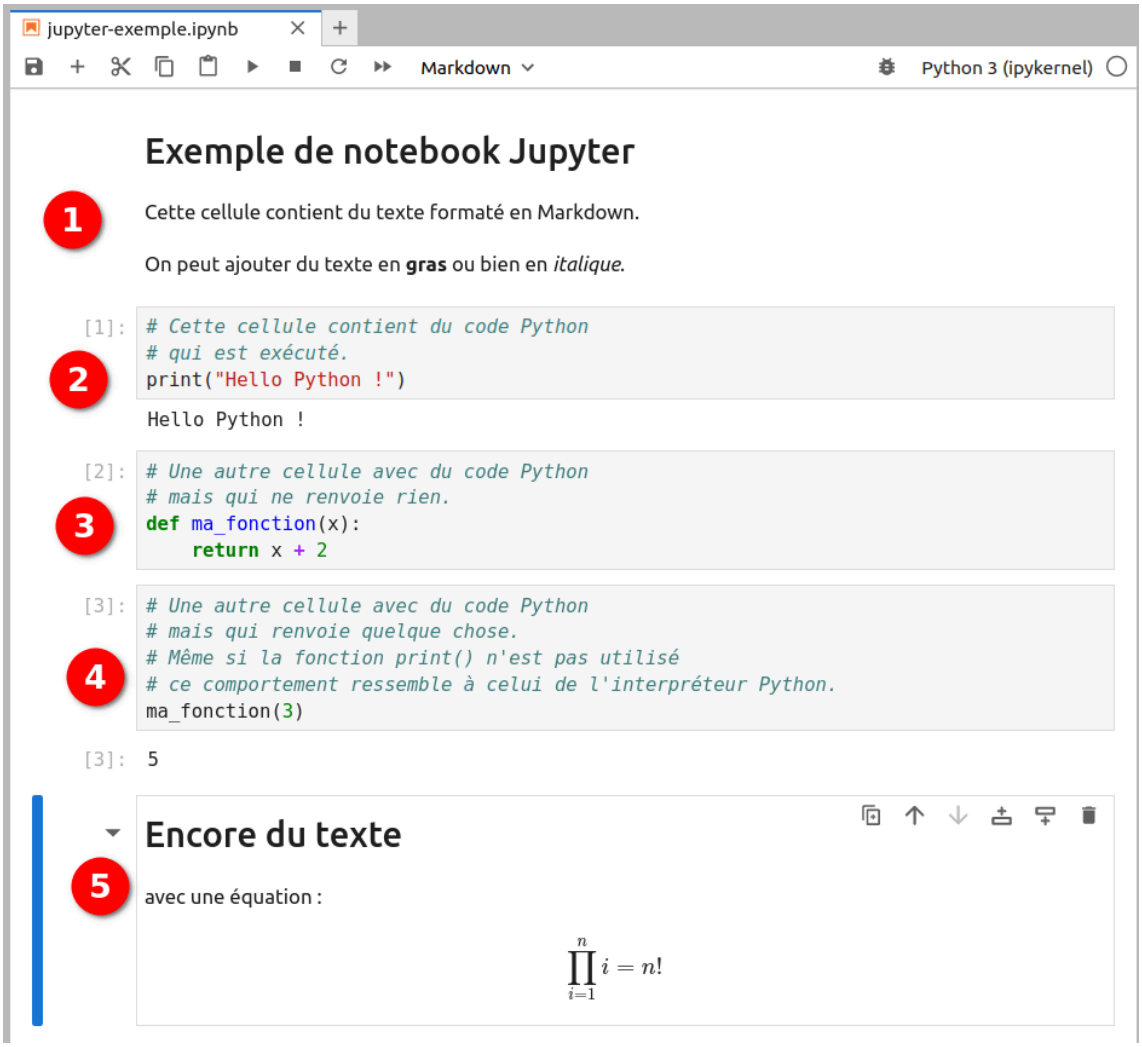


Figure 1. – Illustration du principe d'un *notebook* `Jupyter`. Source: Cours de Python pour les sciences de la vie (Paris 18) par Patrick Fuchs et Pierre Poulain

A l'origine, le logiciel `Jupyter` était le seul offrant ces fonctionnalités interactives. Il existe maintenant d'autres manières de bénéficier des avantages du *notebook* tout en ayant un IDE aux fonctionnalités plus complètes que `Jupyter`. C'est pour cette raison qu'il est plus pratique, en 2026, d'utiliser `VSCode`, un éditeur de code généraliste mais proposant d'excellentes fonctionnalités en `Python`, que directement `Jupyter`. Déjà très pratique pour utiliser des *notebooks*, lorsque vous utiliserez `Python` par le biais des scripts, un vrai IDE comme `VSCode` vous sera essentiel.

> **i** Pour en savoir plus
>
> Pour en savoir plus sur l'utilisation des *notebooks* dans VSCode, se référer à la documentation officielle.
>
> https://code.visualstudio.com/assets/docs/datascience/jupyter/notebook-run-in-section.mp4

3 Utilisation des *notebooks* dans le cadre de ce cours

La meilleure manière d'apprendre est la pratique. Pour permettre celle-ci, tous les chapitres de ce cours seront exécutables et modifiables par le biais de *notebooks*. Les boutons suivants permettent d'ouvrir ce chapitre sous forme de *notebook* dans différents environnements:

1. Sur `Github` , seulement pour visualiser et télécharger ceux-ci car `Github` n'est pas un environnement de développement et d'utilisation des *notebooks* ;
2. Sur le `SSPCloud`, un *cloud* moderne développé par l'Insee et mis à disposition gratuitement aux agents publics, aux étudiants, aux chercheurs et aux agents des instituts de statistiques publiques européens. Comme ceci est mentionné dans l'encadré dédié, c'est le point d'entrée aux *notebooks* recommandé pour toutes les personnes ayant la possibilité de l'utiliser. Les *notebooks* peuvent y être ouverts via `VSCode` et bénéficient d'un accès à une ligne de commande avec les droits adéquats pour l'installation de *package*. A ces caractéristiques déjà désirables pour découvrir `Python`, s'ajoutent d'autres fonctionnalités utiles pour l'apprentissage en continu et qui sont plus explorées en troisième année dans le cours de « Mise en production de projets data science »: accès à de la GPU gratuitement, interfaçage avec d'autres technologies *cloud* comme un système de stockage objet, etc.
3. Google Colab est un service en ligne gratuit, basé sur l'interface de `Jupyter`, qui permet d'accéder à des ressources `Python` exécutées par les serveurs de Google.
4. Github codespaces permet également d'ouvrir les *notebooks*, mais cette solution n'a pas été testée pour ce cours : le résultat n'est donc pas garanti.

> ! Environnement recommandé pour ce cours
>
> Pour les agents de la fonction publique, ou les élèves des écoles partenaires, il est recommandé de privilégier le bouton `SSPCloud` qui est une infrastructure *cloud* moderne, puissante et flexible développée par l'Insee et accessible à l'url https://datalab.sspcloud.fr[6].

4 Exercice pour découvrir les fonctionnalités basiques d'un *notebook*

Tous les chapitres de ce cours sont pensés sous la forme d'un fil conducteur visant à répondre à un besoin pratique avec des exercices intermédiaires faisant office de jalons. Ils sont facilement identifiables sous la forme suivante:

> Tip
>
> Un exemple de boite exercice

L'exercice suivant a pour but de vous familiariser avec l'utilisation des *notebooks* si vous ne connaissez pas ce type de document. Il illustre des fonctionnalités de base comme l'écriture de code, l'exécution de cellules, l'ajout de texte, et la visualisation de données.

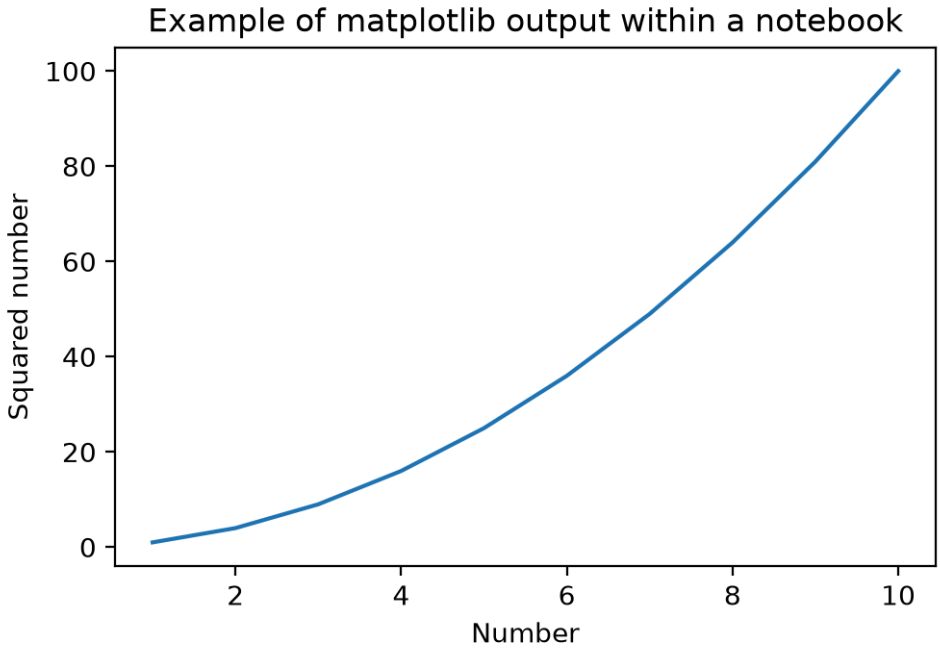
[6] Pour les utilisateurs de cette infrastructure, les *notebooks* de ce cours sont également listés, parmi de nombreuses autres ressources de qualité, sur la page `Formation`

Pour cela, ouvrir ce chapitre dans un environnement adapté aux *notebooks*:[7]

Exercice 1

1. Sous cet exercice, créer une cellule de code. Ecrivez un code `Python` qui affiche la phrase : *« Bienvenue dans un notebook ! »* puis exécutez la cellule. Modifiez votre code et re-exécutez.
2. En cherchant sur internet, ajoutez une nouvelle cellule et changez son type en « Markdown ». Dans cette cellule, écrivez un court texte comportant les éléments suivants:
   - Un petit bout en italique
   - Une liste non ordonnée
   - Un titre de niveau 2 (équivalent du `<h2>` en HTML ou `\subsection` en $\LaTeX$)
   - Une équation
3. Créez une cellule de code, n'importe où dans le document. Créez une liste de nombres entiers allant de 1 à 10 sous le nom `numbers`. Affichez cette liste.
4. Créez une nouvelle cellule de code. Utilisez le code sous cet exercice pour générer une figure.

La figure obtenue à la fin de l'exercice ressemblera à celle-ci:



5 Comment résoudre les erreurs ?

La survenue d'erreurs est tout à fait naturelle et attendue lors de l'apprentissage (et même après !) d'un langage informatique. La résolution de ces erreurs est vraiment l'occasion de comprendre comment fonctionne le langage et de devenir autonome dans sa pratique de celui-ci.

Voici une proposition d'étapes à suivre (dans cet ordre) pour résoudre une erreur :

1. **Bien lire les *logs***, i.e. les sorties renvoyées par `Python` en cas d'erreur. Souvent, elles sont informatives et peuvent contenir directement la réponse.

[7] Pour utiliser https://datalab.sspcloud.fr si vous y êtes éligibles, vous devez vous créer un compte.

2. **Chercher sur internet** (de préférence en Anglais et sur `Google`). Par exemple, donner le nom de l'erreur et une partie informative du message d'erreur renvoyé par `Python` permet généralement de bien orienter les résultats vers ce que l'on cherche.
3. Souvent, la **recherche amènera vers le forum Stackoverflow**, destiné à cet usage. Si l'on ne trouve vraiment pas la réponse à son problème, on peut poster sur `Stackoverflow` en détaillant bien le problème rencontré de sorte à ce que les utilisateurs du forum puissent le reproduire et trouver une solution.
4. Les **documentations officielles** (de `Python` et des différents packages) sont souvent un peu arides, mais généralement exhaustives. Elles permettent notamment de bien comprendre la manière d'utiliser les différents objets. Par exemple pour les fonctions : ce qu'elles attendent en entrée, les paramètres et leur type, ce qu'elles renvoient en sortie, etc.
5. Les IA assistantes de code (`ChatGPT`, `Claude`) peuvent être d'une grande aide. En faisant attention à leur donner des instructions adaptées et en vérifiant le code produit pour éviter les hallucinations, on peut gagner beaucoup de temps grâce à celles-ci. Attention néanmoins à ne pas devenir fainéants en passant systématiquement par l'étape 5: vous risquez de devenir captif de ces outils sans avoir appris l'autonomie.

Exercice 2: apprendre en se trompant

Corriger la cellule ci-dessous pour qu'elle ne produise plus d'erreur

```
pd.DataFrame(x)
```

```
NameError: name 'pd' is not defined
␛[31m---------------------------------------------------------------------------␛[39m
␛[31mNameError␛[39m                                 Traceback (most recent call last)
␛[36mCell␛[39m␛[36m ␛[39m␛[32mIn[4]␛[39m␛[32m, line 1␛[39m
␛[32m----> ␛[39m␛[32m1␛[39m pd.DataFrame(x)

␛[31mNameError␛[39m: name 'pd' is not defined
```

6 Continuer à se former après ce cours

6.1 Contenu sur ce site

Ce cours est une introduction à la *data science* avec `Python`. La majorité du contenu de celui-ci est donc pensée pour un public qui découvre le sujet ou désire découvrir une thématique plus précise dans ce domaine, par exemple la cartographie ou le *machine learning*.

Cependant, ce cours est aussi le fruit de mon expérience passée de bientôt une décennie 👴 à faire du `Python` sur des sources de données, des infrastructures et des problématiques variées: il est donc assez éditorialisé (« *opinionated* » diraient les anglo-saxons) pour mettre en avant certaines exigences attendues des *data scientists* mais aussi vous éviter de vous fourvoyer dans les mêmes eaux tumultueuses que moi par le passé.

Ce cours propose aussi du contenu permettant d'aller au-delà des premiers mois d'apprentissage. Tout le contenu de ce site web n'est pas enseigné, certaines sections d'approfondissement voire certains chapitres ont vocation à servir à l'apprentissage continu et peuvent être consommés plusieurs mois après la découverte de ce cours.

pythonds.linogaliana.fr évolue constamment pour tenir compte de l'écosystème `Python` mouvant. Les *notebooks* resteront disponibles au-delà du semestre d'enseignement.

6.2 La veille technique

L'écosystème riche et foisonnant de `Python` a comme contrepartie qu'il faut rester attentif à ses évolutions pour ne pas voir son capital humain vieillir et ainsi devenir *has-been*. Alors qu'avec des langages monolithiques comme `SAS` ou `Stata` on pouvait se permettre de ne faire de veille technique mais seulement consulter la documentation officielle, avec `Python` ou `R` c'est impossible. Ce cours lui-même est en évolution continue, ce qui est assez exigeant 😅, pour épouser les évolutions de l'écosystème.

Les réseaux sociaux comme `Linkedin` ou les agrégateurs de contenu comme `medium` ou `towardsdatascience` proposent des *posts* de qualité hétérogène mais il n'est pas inutile d'avoir une veille technique continue sur ces plateformes : au bout d'un certain temps, cela peut permettre de dégager les nouvelles tendances. Le site `realpython` propose généralement de très bon posts, complets et pédagogiques. `Github` peut être utile pour la veille technique: en allant voir les projets à la mode, on peut voir les tendances qui émergeront prochainement.

En ce qui concerne les ouvrages papiers, certains sont de très bonne qualité. Cependant, il convient de faire attention à la date de mise à jour de ceux-ci : la vitesse d'évolution de certains éléments de l'écosystème peut les périmer très rapidement. Il est généralement plus utile de bénéficier d'un *post* non exhaustif mais plus récent qu'un ouvrage complet.

> Les newsletters utiles
>
> Il existe de nombreuses *newsletters* très bien faites pour suivre régulièrement les évolutions de l'écosystème de la *data science*. C'est, pour ma part, ma principale source d'informations fraiches avec les réseaux sociaux.
>
> S'il ne fallait s'abonner qu'à une *newsletter*, la plus importante à suivre est celle de Andrew Ng, *« The batch »*. Réflexions sur les avancées académiques des réseaux de neurone, évolution de l'écosystème logiciel et institutionnel, cette *newsletter* est une excellente *food for thoughts*.
>
> La *newsletter* de Christophe Bleffari à destination des *data engineers* mais qui intéressera également beaucoup les *data scientists* présente souvent du très bon contenu.
>
> La *newsletter* de Rami Krispin (*data scientist à Apple*) est également très utile, notamment quand on travaille régulièrement non seulement avec `Python` mais aussi avec et `Quarto`, le logiciel de publication reproductible.
>
> Assez technique, les vidéos d'Andrej Karpathy sont très instructives pour comprendre le fonctionnement des modèles de langage à l'état de l'art. De même, le contenu produit par Sebastian Raschka permet de connaître les dernières avancées dans la recherche sur le sujet.
>
> Les *newsletters* généralistes de *Data Elixir* et Alpha Signal permettent de connaître les dernières nouveautés. Dans le domaine de la dataviz, celles de *DataWrapper* permettent de bénéficier d'un contenu accessible sur le sujet.

Informations additionnelles

> Environnement Python
>
> Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
    "markdown≥3.8",
    "nbclient≥0.10.0",
    "nbformat≥5.10.4",
    "nltk≥3.9.1",
    "openpyxl≥3.1.5",
    "pandas≥3.0",
    "pip≥25.1.1",
    "plotly≥6.1.2",
    "plotnine≥0.15",
    "polars≥1.8.2",
    "py7zr≥1.1.3",
    "pyarrow≥17.0.0",
    "pynsee≥0.1.8",
    "python-dotenv≥1.0.1",
    "python-frontmatter≥1.1.0",
    "pywaffle≥1.1.1",
    "requests≥2.32.3",
    "scikit-image≥0.24.0",
    "scikit-learn≥1.8.0",
    "scipy≥1.13.0",
    "seaborn≥0.13.2",
    "selenium<4.39.0",
    "spacy≥3.8.4",
    "webdriver-manager≥4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
```

```
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.